

UNIT 4: UNSUPERVISED LEARNING, ENSEMBLE METHODS & PREPROCESSING

Course Pattern: TE IT - 2024 Pattern (SPPU)	Academic Term: Semester V (2026)
Status: Publication-Grade Theory Guide	Course Code: PCC-302-IT

Syllabus Mapping & Unit Structure

The following table outlines the exam-oriented curriculum mapping and priority weightage for Unit 4 based on SPPU end-semester examination trends.

Syllabus Topic	In-Depth Exam Coverage	Priority / Weightage
Data Preprocessing & Cleaning	Data cleaning, Handling missing values (Statistical vs k-NN Imputation), Feature scaling (Min-Max, Standardization), fit() vs transform(), Data leakage prevention.	High Priority (6-8 Marks Qs)
Feature Engineering & Reduction	Curse of Dimensionality, Filter/Wrapper/Embedded feature selection, Pearson Correlation Coefficient, Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA).	High Priority (8-10 Marks Qs)
Partitioning Clustering	Unsupervised clustering taxonomy, k-Means algorithm, Inertia / WCSS, Elbow method, Silhouette analysis, k-Medoids (PAM), Distance metrics.	High Priority (8-10 Marks Qs)
Hierarchical Clustering	Agglomerative vs Divisive, Dendrograms, Linkage criteria (Single, Complete, Average, Ward's Linkage), Connectivity constraints.	High Priority (6-8 Marks Qs)
Association Rule Mining	Market basket analysis, Support, Confidence, Lift, Apriori algorithm, Candidate generation and pruning.	Medium-High Priority (6-8 Marks Qs)
Ensemble Learning & Bagging	Condorcet's Theorem, Bias-variance reduction, Bootstrap Aggregation (Bagging), Out-of-Bag (OOB) error, Random Forest (Random feature subspaces, Tree decorrelation).	High Priority (8-10 Marks Qs)
Boosting & Stacking	Sequential sample reweighting, AdaBoost algorithm formulation, Gradient Boosted Trees (GBM), Stacked Generalization (Meta-learning, Blending, Out-of-fold cross validation).	High Priority (8-10 Marks Qs)

Data Preprocessing, Cleaning & Feature Engineering

Data Preprocessing Workflow & Core Challenges

Real-world datasets gathered from sensors, transactional databases, and logs are inevitably dirty, incomplete, noisy, and unstandardized. Data preprocessing transforms raw, imperfect data into a structured mathematical representation suitable for machine learning algorithms.

1. **Incomplete Data (Missing Values):** Caused by sensor malfunctions, optional user survey fields, or database merge anomalies.
2. **Noisy Data (Errors & Outliers):** Caused by human data entry errors, transmission faults, or extreme anomalous events.
3. **Inconsistent & Redundant Data:** Discrepancies in naming conventions, duplicated entries, or highly correlated features (multicollinearity).
4. **Scale Disparities:** Numeric features measured on wildly partialing units (e.g., Annual Salary in thousands vs. Age in years), which causes distance-based models (k-NN, SVM, k-Means) to become biased toward large-magnitude features.

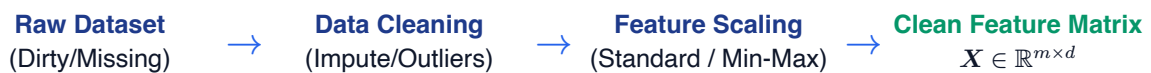


Figure 4.1: Machine Learning Data Preprocessing and Feature Transformation Pipeline

Handling Missing Values in Datasets

Missing data mechanisms are statistically classified into **Missing Completely at Random (MCAR)**, **Missing at Random (MAR)**, and **Missing Not at Random (MNAR)**.

1. **Listwise Deletion (Row Deletion):** Dropping any observation that contains one or more missing feature values.
 - **Advantage:** Simple and avoids introducing synthetic imputation bias.
 - **Disadvantage:** Drastically reduces dataset sample size and discards valid information if missingness is widespread.
2. **Statistical Univariate Imputation:** Replacing missing values with central tendency statistics computed over observed entries:
 - **Mean Imputation:** Replaces missing values with feature mean $\mu = \frac{1}{N} \sum x_i$ (suitable for normally distributed symmetric numeric data).
 - **Median Imputation:** Replaces missing values with 50th percentile (robust to extreme outliers and skewed distributions).
 - **Mode Imputation:** Replaces missing values with the most frequent category (mandatory for discrete nominal/categorical features).
3. **k-Nearest Neighbors (k-NN) Imputation:**
 - Identifies the k most similar complete instances in the dataset based on available non-missing features using Euclidean or Gower's distance.
 - Imputes the missing attribute by calculating the mean or distance-weighted average among the k identified neighbors:

$$\hat{x}_{i,j} = \frac{\sum_{n \in N_{k(i)}} w_n \cdot x_{n,j}}{\sum_{n \in N_{k(i)}} w_n}, \quad w_n = \frac{1}{d(x_i, x_n) + \varepsilon}$$

- **Advantage:** Captures multidimensional feature correlations far better than univariate mean/median imputation.
- **Disadvantage:** Computationally expensive ($\mathcal{O}(m \times d)$) during inference on large-scale datasets.

Technique	Core Mechanism	Key Advantages	Primary Limitations
Listwise Deletion	Delete entire sample if any feature is null.	Fast, preserves original feature distributions.	Severe data loss, introduces bias if not MCAR.
Mean / Median	Replace nulls with column arithmetic mean/median.	Fast, deterministic, preserves sample size.	Distorts feature variance and inter-feature covariance.
Mode Imputation	Replace nulls with most frequent discrete class.	Applicable to categorical and text attributes.	Amplifies class dominance and reduces entropy.
k-NN Imputation	Impute using distance-weighted nearest neighbors.	Preserves multi-attribute correlation patterns.	High time complexity $\mathcal{O}(m^2)$, sensitive to outliers.

Feature Scaling & Normalization

Feature scaling normalizes the dynamic range of independent variables to ensure equal gradient updates and distance calculations.

1. Min-Max Normalization (Feature Scaling to Range $[0, 1]$):

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

- Rescales all numeric values strictly into bounded interval $[0, 1]$ (or $[a, b]$).
- Highly sensitive to outliers, as extreme values compress the vast majority of inlier data into a tiny sub-interval.

2. Standardization (Z-Score Normalization):

$$z = \frac{x - \mu}{\sigma}$$

where μ is the empirical sample mean and σ is the sample standard deviation.

- Centers data around mean $\mu = 0$ with unit variance $\sigma^2 = 1$.
- Does not bind features to a fixed minimum/maximum range, making it robust to outliers.
- Mandatory for algorithms assuming Gaussian error distributions (Linear/Logistic Regression, PCA, SVM).

Scikit-Learn API Design & Preventing Data Leakage

In production machine learning pipelines, preprocessing transformations are strictly divided into parameter estimation and application:

1. **fit() Method:** Computes and stores the internal transformation parameters (e.g., mean μ , standard deviation σ , min, max) **strictly from the training dataset only**.
2. **transform() Method:** Applies the learned transformation parameters to scale or impute target feature matrices.
3. **fit_transform() Method:** Combines fit() and transform() in a single optimized pass on training data.
4. **Data Leakage Prevention Rule: NEVER** call fit() or fit_transform() on the test dataset. Doing so leaks test set distribution statistics into the model training pipeline, resulting in over-optimistic evaluation scores that fail in production deployment.

$$X_{\text{train_scaled}} = \text{scaler.fit_transform}(X_{\text{train}})$$

$$X_{\text{test_scaled}} = \text{scaler.transform}(X_{\text{test}})$$

Feature Selection Techniques & Pearson Correlation

Feature selection reduces model complexity by selecting the most informative subset of original features without geometric alteration.

1. **Filter Methods:** Select features using statistical properties independent of model training:
 - **Pearson's Correlation Coefficient (r):** Measures linear bivariate association between continuous features X and Y :

$$r_{X,Y} = \frac{\sum_{i=1}^m (x_i - |x|)(y_i - |y|)}{\sqrt{\sum_{i=1}^m (x_i - |x|)^2} \sqrt{\sum_{i=1}^m (y_i - |y|)^2}}$$

- Value ranges from -1.0 (perfect negative linear correlation) to $+1.0$ (perfect positive linear correlation), with 0 indicating no linear correlation.
 - **Feature Filtering Strategy:** Retain features with high correlation to the target variable Y , and eliminate redundant features with high mutual inter-correlation ($|r| > 0.85$).
 - **Chi-Square (χ^2) Test:** Evaluates independence between categorical features and target class labels.
 - **ANOVA F -Test:** Tests variance partialerences of numeric features across multiple discrete classes.
2. **Wrapper Methods:** Use a predictive model as an evaluation engine to search feature combinations (e.g., **Forward Feature Selection**, **Backward Elimination**, **Recursive Feature Elimination - RFE**). Computationally expensive but highly accurate.
 3. **Embedded Methods:** Perform feature selection automatically during model training (e.g., **Lasso L_1 Regression**, **Decision Tree Gini Importance**).

Source: Alpaydin, Ch 6, p. 109-125 | Source: Marsland, Ch 6, p. 129-148 | Source: Müller & Guido, Ch 3, p. 131-155

Dimensionality Reduction: PCA and LDA

The Curse of Dimensionality

As the number of features (dimensions d) increases, the volume of the feature space grows exponentially ($V \propto r^d$), causing the available training data points to become extremely sparse.

1. **Distance Concentration:** In high-dimensional spaces, the Euclidean distance between any two randomly chosen points converges to the same value ($\lim_{d \rightarrow \infty} \frac{d_{\max} - d_{\min}}{d_{\min}} = 0$), destroying distance-based discrimination in k-NN, SVM, and k-Means.
2. **Combinatorial Explosion & Overfitting:** High dimensions require exponentially more training samples to maintain sample density, leading to severe overfitting.
3. **Remedy:** Dimensionality reduction projects high-dimensional data $\mathbb{R}^d$ into a lower-dimensional subspace $\mathbb{R}^k$ ($k \ll d$) while preserving maximal statistical variance or class separability.

Principal Component Analysis (PCA)

Principal Component Analysis is an **unsupervised linear dimensionality reduction technique** that finds orthogonal axes (Principal Components) along which data variance is maximized.

Principal Component Analysis (PCA)

- **Unsupervised** (ignores class labels)
 - Finds axis w_1 maximizing data variance
 - Minimizes reconstruction error $\|x - \hat{x}\|^2$

Linear Discriminant Analysis (LDA)

- **Supervised** (uses class labels)
 - Maximizes between-class variance (S_B)
 - Minimizes within-class variance (S_W)

Figure 4.2: PCA (Max Variance Projection) vs LDA (Max Class Separability Projection)

1. Mathematical Derivation & Step-by-Step Algorithm:

- **Step 1 (Standardization):** Center data matrix $X \in \mathbb{R}^{m \times d}$ so each feature has mean zero ($\mu = 0$).
- **Step 2 (Covariance Matrix Calculation):** Compute the $d \times d$ sample covariance matrix Σ :

$$\Sigma = \frac{1}{m-1} X^T X$$

- **Step 3 (Eigenvalue Decomposition):** Solve the characteristic eigenvalue equation:

$$\Sigma v_j = \lambda_j v_j$$

where λ_j is the j -th eigenvalue and v_j is the corresponding j -th eigenvector (orthogonal principal component direction).

- **Step 4 (Sorting & Component Selection):** Sort eigenvalues in descending order: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$.
- **Step 5 (Explained Variance Ratio):** Select top k eigenvectors explaining target variance threshold (e.g., 95%):

$$\text{Explained Variance Ratio} = \frac{\sum_{j=1}^k \lambda_j}{\sum_{j=1}^d \lambda_j} \geq 0.95$$

- **Step 6 (Projection):** Form projection matrix $W = [v_1, v_2, \dots, v_k] \in \mathbb{R}^{d \times k}$ and transform data:

$$X_{\text{projected}} = XW \in \mathbb{R}^{m \times k}$$

2. **Geometric Interpretation:** First principal component (PC_1) captures the direction of greatest data spread; second principal component (PC_2) is strictly orthogonal ($v_1^T v_2 = 0$) and captures the next highest variance.

Linear Discriminant Analysis (LDA)

Linear Discriminant Analysis (Fisher's LDA) is a **supervised dimensionality reduction and classification technique** that finds a linear combination of features that maximizes class separation.

1. **Fisher's Criterion Formulation:** LDA seeks projection vector w that maximizes the ratio of between-class scatter to within-class scatter:

$$J(w) = \frac{w^T S_B w}{w^T S_W w}$$

2. **Scatter Matrices Definitions:**

- **Within-Class Scatter Matrix (S_W):** Measures dispersion of samples around their respective class centroids:

$$S_W = \sum_{c=1}^C \sum_{x \in D_c} (x - \mu_c)(x - \mu_c)^T$$

- **Between-Class Scatter Matrix (S_B):** Measures dispersion of class centroids around the global mean μ :

$$S_B = \sum_{c=1}^C N_c (\mu_c - \mu)(\mu_c - \mu)^T$$

3. **Optimal Projection Solution:** Maximizing $J(w)$ reduces to solving the generalized eigenvalue problem:

$$S_W^{-1} S_B w = \lambda w$$

4. **Dimensionality Constraint:** Since S_B is the sum of C rank-1 matrices of which only $C - 1$ are linearly independent, the rank of S_B is at most $C - 1$. Therefore, LDA can project data to at most $k \leq C - 1$ dimensions (where C is the number of distinct classes).

Parameter	Principal Component Analysis (PCA)	Linear Discriminant Analysis (LDA)
Learning Paradigm	Unsupervised (ignores target class labels).	Supervised (requires target class labels y_i).
Optimization Objective	Maximize total feature variance / minimize reconstruction error.	Maximize between-class separability relative to within-class spread.
Mathematical Engine	Eigen decomposition of sample Covariance Matrix Σ .	Generalized eigen decomposition of $S_W^{-1} S_B$.
Maximum Dimensions	Up to original number of features $k \leq d$.	Strictly bounded by number of classes $k \leq C - 1$.
Primary Application	Feature compression, noise filtering, data visualization.	Supervised dimensionality reduction, pre-classification projection.

Source: Alpaydin, Ch 6, p. 109-130 | Source: Marsland, Ch 6, p. 129-152 | Source: Müller & Guido, Ch 3, p. 140-155

Unsupervised Clustering & Association Rule Mining

Taxonomy of Clustering Algorithms

Clustering is the unsupervised grouping of unlabeled objects into clusters such that objects within the same cluster possess high intra-cluster similarity, while objects across partialing clusters exhibit high inter-cluster dissimilarity.

1. **Partitioning Methods:** Construct K disjoint partitions directly from data (e.g., k -Means, k -Medoids / PAM).
2. **Hierarchical Methods:** Create a nested decomposition of instances represented as a tree hierarchy (e.g., Agglomerative, Divisive).
3. **Density-Based Methods:** Form clusters based on spatial data density, discovering arbitrary-shaped clusters and filtering noise (e.g., DBSCAN, OPTICS).
4. **Model-Based / Grid-Based Methods:** Assume data is generated from a mixture of underlying probability distributions (e.g., Gaussian Mixture Models - GMM via EM algorithm).

The k -Means Clustering Algorithm

k -Means is an iterative centroid-based partitional clustering algorithm that partitions m observations into k clusters.



Figure 4.3: Iterative Centroid Convergence and Voronoi Partitioning in k -Means

1. **Objective Function (Within-Cluster Sum of Squares / Inertia):**

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

where μ_k is the mathematical centroid (mean) of cluster C_k .

2. **Lloyd's Iterative Algorithm:**

- **Initialization:** Select k initial cluster centroids $\mu_1, \mu_2, \dots, \mu_k$ in feature space.
- **Step 1 (Assignment Step):** Assign each observation x_i to its nearest centroid according to Euclidean distance:

$$c^{(i)} = \arg \min_k \|x_i - \mu_k\|^2$$

- **Step 2 (Update Step):** Recompute the centroid of each cluster as the arithmetic mean of all assigned points:

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

- **Step 3 (Convergence Check):** Repeat Steps 1 and 2 until centroid positions stabilize ($\|\mu_k^{(t+1)} - \mu_k^{(t)}\| < \epsilon$), point assignments cease changing, or maximum iterations are reached.

3. **Determining Optimal k :**

- **Elbow Method:** Plots inertia J against values of k . The “elbow” bend represents the point of diminishing marginal returns.
- **Silhouette Coefficient (s_i):** Combines mean intra-cluster distance (a_i) with mean nearest-cluster distance (b_i):

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

- Value ranges from -1.0 (incorrect clustering) to $+1.0$ (highly dense and well-separated clusters), with 0 indicating overlapping clusters.

4. **Limitations of Standard k -Means:**

- Assumes spherical, equal-variance clusters; fails on elongated, concentric, or irregular manifold shapes.

- Highly sensitive to initial random centroid placement (can get trapped in poor local optima; mitigated by **k-Means++**).
- Sensitive to extreme outliers because the arithmetic mean is pulled by anomalous values.

The *k*-Medoids Clustering Algorithm (PAM)

k-Medoids is a partitional clustering algorithm that selects **actual data points as cluster exemplars (medoids)** rather than virtual arithmetic means.

1. Partitioning Around Medoids (PAM) Algorithm:

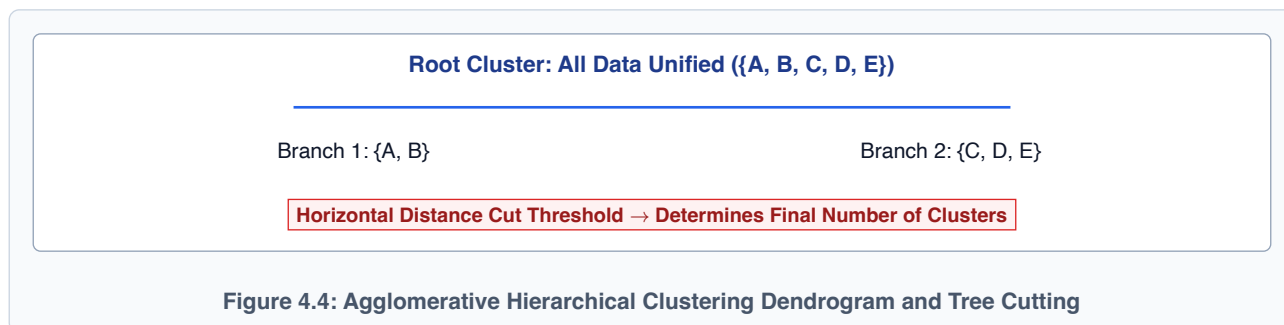
- Initialize *k* actual data points as medoids.
- Assign each non-medoid point to the closest medoid using Manhattan or arbitrary distance metrics.
- For each medoid *m* and each non-medoid point *p*: swap *m* and *p* and calculate the total cost variation (ΔS). If $\Delta S < 0$, keep the swap.

2. Key Advantage: Extreme robustness to noise and outliers because medoids are central actual members rather than outlier-sensitive arithmetic means.

Parameter	k-Means Clustering	k-Medoids Clustering (PAM)
Center Representation	Arithmetic mean (virtual centroid point).	Actual exemplar data point (medoid).
Distance Metric	Strictly Euclidean distance (squared error minimization).	Flexible (Manhattan, Minkowski, Cosine, arbitrary).
Sensitivity to Outliers	High (extreme outliers distort arithmetic means).	Low (highly robust; outliers rarely chosen as medoids).
Computational Complexity	$\mathcal{O}(tkmd)$ (Linear and scalable to huge datasets).	$\mathcal{O}(tk(m-k)^2)$ (Computationally heavy for large <i>m</i>).

Hierarchical Clustering & Linkage Metrics

Hierarchical clustering builds a nested hierarchy of clusters visualized as a tree structure called a **Dendrogram**.



1. Agglomerative (Bottom-Up) vs. Divisive (Top-Down):

- **Agglomerative:** Starts with *m* singleton clusters; iteratively merges the two closest clusters until 1 unified cluster remains ($\mathcal{O}(m^3)$ or $\mathcal{O}(m^2 \log m)$).
- **Divisive:** Starts with 1 all-inclusive root cluster; recursively splits clusters top-down.

2. Linkage Criteria (Inter-Cluster Distance Measures):

- **Single Linkage (MIN / Nearest Neighbor):** Distance between the two closest points across clusters:

$$d_{\text{Single}(A,B)} = \min_{x \in A, z \in B} d(x, z)$$

- **Characteristic:** Handles non-elliptical shapes well; highly susceptible to the **chaining effect** (forming long straggly clusters connected by single noise points).
- **Complete Linkage (MAX / Farthest Neighbor):** Distance between the two most distant points:

$$d_{\text{Complete}}(A, B) = \max_{x \in A, z \in B} d(x, z)$$

- **Characteristic:** Highly robust to chaining; forces compact, spherical clusters of equal diameter.
- **Average Linkage (UPGMA):** Average distance between all pairwise combinations:

$$d_{\text{Average}}(A, B) = \frac{1}{|A| \cdot |B|} \sum_{x \in A} \sum_{z \in B} d(x, z)$$

- **Ward's Linkage (Minimum Variance):** Merges cluster pair that minimizes the total within-cluster variance increase (minimum increase in SSE).

Association Rule Mining & The Apriori Algorithm

Association Rule Mining discovers interesting relational dependencies and co-occurrence patterns among items in transactional databases.

1. **Core Association Metrics:** For rule $X \rightarrow Y$ where itemsets $X, Y \subseteq I$ and $X \cap Y = \emptyset$:

- **Support:** Frequency of occurrence of itemset in database:

$$\text{Support}(X \rightarrow Y) = P(X \cup Y) = \frac{\text{Transactions containing } X \cup Y}{\text{Total Transactions } N}$$

- **Confidence:** Conditional probability that a transaction contains Y given it contains X :

$$\text{Confidence}(X \rightarrow Y) = P(Y | X) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

- **Lift:** Measures the strength of a rule over random co-occurrence independence:

$$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)} = \frac{P(X \cup Y)}{P(X) \cdot P(Y)}$$

- Lift > 1.0: Positive correlation (items appear together more often than chance).
- Lift = 1.0: Statistical independence.
- Lift < 1.0: Negative correlation (substitution effect).

2. **The Apriori Algorithm & Downward Closure Property:**

- **Apriori Principle:** Any subset of a frequent itemset must also be frequent ($\forall S \subseteq X$, if $\text{Support}(X) \geq \text{min_sup}$, then $\text{Support}(S) \geq \text{min_sup}$).
- **Pruning Property:** If an itemset is infrequent, all of its supersets are immediately pruned without database scanning.
- **Level-wise Iteration:** Generates candidate k -itemsets (C_k) from frequent $(k-1)$ -itemsets (L_{k-1}), prunes infrequent candidates, and scans database until no larger frequent itemsets can be formed.

Source: Alpaydin, Ch 7, p. 137-160 | Source: Marsland, Ch 14, p. 281-305 | Source: Müller & Guido, Ch 3, p. 165-200

Ensemble Learning: Bagging, Random Forest, Boosting & Stacking

Foundational Concepts of Ensemble Learning

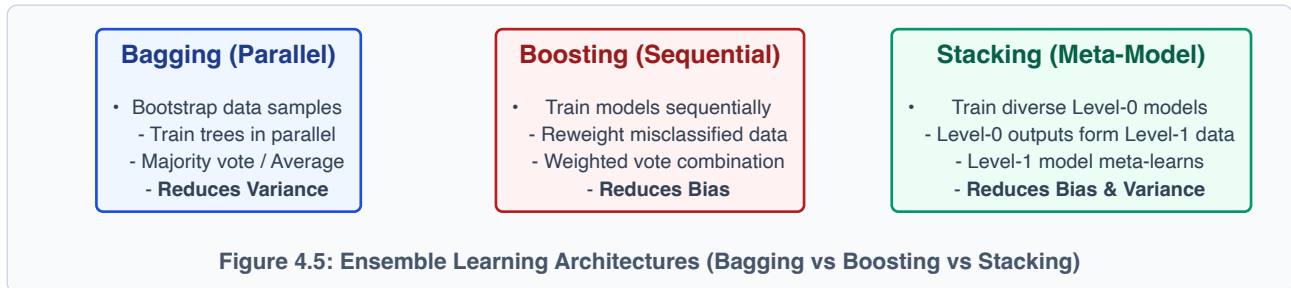
Ensemble learning combines the predictions of multiple individual base estimators (weak learners) to produce a unified meta-model with superior generalization, robustness, and stability.

1. **Condorcet's Jury Theorem:** If each individual base classifier operates with an independent probability $p > 0.5$ of being correct, the cumulative probability that a majority vote of M diverse classifiers is correct approaches 1.0 as $M \rightarrow \infty$.
2. **Base Learners:**
 - **Weak Learner:** An algorithm that performs only slightly better than random guessing (e.g., shallow decision trees / decision stumps with accuracy $\approx 55\%$).

- **Strong Learner:** An algorithm with arbitrarily high accuracy achieved through ensemble combination.

3. Homogeneous vs. Heterogeneous Ensembles:

- **Homogeneous:** Employs identical base algorithm across all learners with varying data subsets (e.g., Random Forest using Decision Trees).
- **Heterogeneous:** Combines fundamentally partialing model architectures (e.g., Stacking k-NN, Logistic Regression, and SVM).



Bootstrap Aggregating (Bagging) & Random Forest

1. Bagging (Bootstrap Aggregation):

- Generates B distinct training sets of size m by sampling from original dataset D **uniformly with replacement** (Bootstrap Sampling).
- Each bootstrap sample contains approximately $1 - \frac{1}{e} \approx 63.2\%$ of original instances; the remaining 36.8% form the **Out-of-Bag (OOB)** validation set.
- Trains an independent base estimator $h_{b(x)}$ on each bootstrap set in parallel.
- Aggregates final predictions via majority voting (classification) or simple averaging (regression):

$$\hat{y} = \frac{1}{B} \sum_{b=1}^B h_{b(x)}$$

- **Primary Impact:** Drastically reduces model **variance** without increasing bias.

2. Random Forest Algorithm: Random Forest enhances Bagging by introducing **Random Feature Subspace Sampling (Feature Bagging)** to de-correlate individual decision trees:

- At each split candidate node in a tree, randomly select a subset of k features from total d features (standard default: $k = \sqrt{d}$ for classification, $k = \frac{d}{3}$ for regression).
- Split node using only the best feature from the chosen k -feature subset.
- Fully grown trees without pruning have low bias; averaging across de-correlated trees minimizes ensemble variance:

$$\text{Var}(\text{Ensemble}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

where ρ is pairwise tree correlation. As $\rho \rightarrow 0$, ensemble variance approaches 0.

3. Feature Importance Metrics:

- **Mean Decrease in Impurity (MDI / Gini Importance):** Total Gini impurity reduction brought by a feature across all trees.
- **Permutation Importance (Mean Decrease Accuracy - MDA):** Measures drop in OOB score when feature values are randomly shuffled.

Boosting Algorithms: AdaBoost & Gradient Boosting

Boosting builds an ensemble sequentially, where each new weak learner is explicitly trained to correct the residual errors made by preceding models.

1. AdaBoost (Adaptive Boosting) Formulation:

- **Step 1:** Initialize sample weights uniformly: $w_i^{(1)} = \frac{1}{m}$ for all $i = 1, \dots, m$.
- **Step 2:** For iteration $t = 1$ to T :
 - Train base classifier $h_{t(x)}$ using sample weights $w^{(t)}$.
 - Compute weighted training error rate:

$$\varepsilon_t = \sum_{i=1}^m w_i^{(t)} \cdot \mathbb{I}(h_{t(x^{(i)})} \neq y^{(i)})$$

- Calculate classifier importance weight α_t :

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_t}{\varepsilon_t} \right)$$

- Update sample weights: increases weights of misclassified instances, decreases weights of correctly classified instances:

$$w_i^{(t+1)} = \frac{w_i^{(t)} \exp(-\alpha_t y^{(i)} h_{t(x^{(i)})})}{Z_t}$$

where Z_t is a normalization factor ensuring $\sum w_i^{(t+1)} = 1$.

- **Step 3 (Final Output):** Compute weighted sign majority:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_{t(x)} \right)$$

2. Gradient Boosting Machines (GBM):

- Fits each new base model directly to the **pseudo-residuals (negative gradient of loss function)** of previous ensemble:

$$r_{i,t} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f(x)=f_{t-1}(x)}$$

- State-of-the-art scalable implementations include **XGBoost** (incorporates second-order Taylor expansion and tree complexity regularization), **LightGBM** (histogram-based bucketing and leaf-wise tree growth), and **CatBoost** (native categorical feature handling).

Stacked Generalization (Stacking)

Stacking is a meta-learning ensemble technique where predictions of heterogeneous base models are used as training features for a meta-classifier.

1. Two-Tier Architecture:

- **Level-0 (Base Estimators):** Diverse algorithms (e.g., Random Forest, SVM, k-NN, Logistic Regression) trained on the original dataset.
- **Level-1 (Meta-Learner):** A secondary model (typically Logistic Regression or Ridge) trained on the prediction matrix of Level-0 models.

2. **Out-of-Fold (OOF) Prediction Pipeline:** To prevent catastrophic target leakage into Level-1, Level-0 predictions are generated strictly using K -Fold Cross Validation: Level-1 features for fold k are predicted by models trained on the other $K - 1$ folds.

Parameter	Bagging (e.g., Random Forest)	Boosting (e.g., AdaBoost/GBM)	Stacking (Meta-Learner)
Training Scheme	Parallel and independent.	Sequential and dependent on errors.	Two-tiered (Base models then Meta-model).
Base Learners	Homogeneous high-variance models (Deep Trees).	Homogeneous high-bias models (Shallow Trees).	Heterogeneous diverse model families (SVM, Trees, kNN).
Aggregation Method	Simple majority vote / arithmetic average.	Weighted majority vote according to α_t .	Meta-model learns optimal combination weights.
Primary Error Goal	Variance reduction (eliminates overfitting).	Bias reduction (builds high accuracy from weak rules).	Both Bias & Variance reduction via diverse architectures.

Parameter	Bagging (e.g., Random Forest)	Boosting (e.g., AdaBoost/GBM)	Stacking (Meta-Learner)
Parallelizability	High (each tree trains independently).	Low (inherently sequential iteration).	High at Level-0, sequential at Level-1.

Source: Alpaydin, Ch 17, p. 419-446 | Source: Marsland, Ch 13, p. 267-280 | Source: Müller & Guido, Ch 2, p. 83-92
| Source: Russell & Norvig, Ch 19, p. 726-735